

Software-as-a-Graph: Dependency-Graph Analysis and Learning for Pre-Deployment Simulated Cascade-Impact Ranking in Publish–Subscribe Systems

Ibrahim Onuralp Yigit^{a,*}, Feza Buzluca^a

^a*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Publish–subscribe middleware decouples components in space and time, obscuring the paths along which failures propagate. Architects need to identify systemically critical components before deployment, when no runtime telemetry exists. We present Software-as-a-Graph (SaG), a framework that models architectures as typed multigraphs and derives an explicit logical dependency graph from configuration manifests. In a pre-registered evaluation under leave-one-scenario-out cross-validation over twelve synthetic architectures, the registered primary contrast between graph transformers and closed-form centrality failed to reach significance ($+0.069$, $p = 0.266$). Instead, training-free architectural afferent coupling on the derived graph—counting an application’s dependents (**InDeg**)—ranks simulated cascade impact at Spearman $\rho = 0.764$, beating closed-form centrality on all twelve held-out architectures ($+0.211$). On models inspired by five open-source systems, transitive reachability reaches $\rho = 0.938$ without training. Graph attention networks on the dependency graph match the count ($\rho = 0.748$), while heterogeneous transformers fail to train stably on the projection. Evaluated against an independent discrete-event queue-flow simulator (I_{dyn}), the dependency count maintains predictive rank correlation without architectural circularity. Dependency counting executes in milliseconds, whereas neural feature extraction takes orders of magnitude longer than direct simulation. These findings provide an empirical benchmark and negative result for graph neural networks in architectural reliability analysis, demonstrating that deriving explicit dependency graphs makes pre-deployment cascade impact rankable with simple structural counts.

Keywords: Dependency graphs, cascading failures, publish–subscribe, graph neural networks, software architecture, dependability, empirical study

1. Introduction

1.1. Motivation and Problem

Large distributed systems increasingly communicate through asynchronous publish–subscribe (pub-sub) middleware: ROS 2 in autonomous driving [1], Apache Kafka in enterprise event streams [2], DDS in cyber-physical systems [3], MQTT in IoT [4], and cloud-native microservices [5, 6]. Pub-sub decouples producers and consumers in space, time and synchronization [7]. Components interact through topics and brokers rather than direct references, and deployment-time Quality-of-Service (QoS) policies govern reliability, durability, priority and deadlines.

The same decoupling hides how failures spread. Publishers and subscribers share no direct link, so outages, head-of-line blocking and backpressure propagate along concealed paths through brokers, shared topics, colocated hosts and shared libraries [8, 9]. These failures take two forms. In *sequential cascades*,

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

a slow subscriber fills a broker queue and gradually starves its publishers [10]. In *simultaneous blasts*, a shared-library crash or host outage takes down every colocated service at once. Neither architecture diagrams nor static call graphs show these mechanisms. The cheapest time to reduce the risk is before deployment, at design and continuous-integration time [11, 12], when no runtime telemetry exists. Architects therefore need to know, from configuration manifests alone, which components, topics and links are systemically critical and why.

Existing practice leaves this gap open, which we call the **Architecture–Code Gap**: a system can have bug-free code in every service and still be fragile through hidden single points of failure or mismatched QoS contracts [13, 14]. Architecture evaluations such as ATAM rely on manual elicitation [15]. Static code analysis inspects services in isolation [16, 17]. Chaos engineering [18] needs a provisioned cluster, while pre-production alternatives like service-level fault injection testing [19] and microservice dependency tracing [20] require runnable execution environments. Homogeneous centrality flattens typed topologies into untyped graphs [21, 22]. Learned models could combine these structural cues, but on their own they produce risk scores without actionable explanations. What is missing is a representation that makes pub-sub failure paths explicit, and evidence on which analyzer to trust on it. This paper shows that once those paths are derived, they can be counted: a component’s dependents on the derived graph rank simulated cascade impact well, before any code is deployed.

1.2. The Software-as-a-Graph (SaG) Approach

Software-as-a-Graph (SaG) is a pre-deployment static system analysis framework for event-driven architectures (Figure 1). It (1) models an architecture as a typed, directed multigraph over Applications, Brokers, Topics, Execution Hosts and Shared Libraries (Section 3.1); (2) derives from it, through publish–subscribe rules, an explicit `DEPENDS_ON` dependency graph that captures both sequential cascades and simultaneous blasts (Section 3.2); (3) ranks components by predicted cascade impact with training-free engines on that graph, graph-learning engines, and hybrid engines in which a learned model corrects a closed-form score (Section 4); and (4) explains flagged components with an ISO/IEC 25010 Reliability–Maintainability (RM) profile (Section 5). Predictors read only the analysis graph. Ground-truth impact comes from independent simulation oracles that run on the raw structural topology (Section 4.4).

The central thesis of this work is that in pre-deployment architectural dependability, deriving an explicit representation of logical dependencies is what makes cascading-failure risk legible: on this derived dependency graph, simple training-free counting of a component’s dependents achieves accuracy that matches or exceeds complex learned or spectral analyzers. Counting a component’s dependents (**InDeg**) ranks simulated cascade impact at $\rho = 0.764$ on held-out architectures, beating closed-form centrality on all twelve folds (+0.211), and counting transitive dependents reaches 0.938 on models inspired by real-world open-source systems. Graph neural networks reading the dependency graph gain +0.08 to +0.11 over reading the raw multigraph, matching the dependency count ($\rho = 0.748$) but failing to exceed it. These findings position the study as an empirical negative result for deep graph learning in architectural dependability, demonstrating that complex analyzers struggle to extract predictive signal beyond direct topological fan-in.

1.3. Research Questions

- **RQ1 (Ranking accuracy):** *How accurately do SaG’s closed-form, learned and hybrid engines rank components by cascading-failure impact on unseen architectures, compared with structural baselines and with dependency counts on the derived graph?*
- **RQ2 (What learning needs):** *What do learned engines need: which graph they read, relation-specific (typed) parameters, or QoS inputs, once model capacity and edge-channel width are matched?*
- **RQ3 (Transfer):** *How well do engines trained on synthetic architectures transfer zero-shot to independently authored models inspired by five open-source systems?*
- **RQ4 (Cost):** *What does the analysis cost at CI/CD time, which stage dominates, and how does it compare with running the simulation directly?*

In our pre-registered analysis plan, the confirmatory primary contrast between graph transformers and closed-form centrality failed to reach significance (+0.069, $p = 0.266$). The dependency counts, the dependency-graph

learners and the derivation checks were evaluated as planned exploratory families (Amendments 7, 9 and 10) outside the 13-contrast confirmatory omnibus. Supplementary Section S24 logs every change as an amendment and maps these four questions onto the five of the registered analysis plan.

1.4. Contributions

Evaluated under leave-one-scenario-out (LOSO) cross-validation over twelve synthetic architectures and zero-shot on five open-source system models, this paper contributes:

1. **A publish–subscribe dependency derivation that formalizes architectural afferent coupling** (Section 3). Rules turn topics, brokers, hosts and shared libraries into explicit logical dependencies that expose sequential cascades and simultaneous blasts. On the derived graph, counting an Application’s dependents (**InDeg**) is identical to its raw 2-hop subscriber count (publish–subscribe afferent coupling / AIS), ranking simulated cascade impact at $\rho = 0.764$ and beating closed-form centrality on all twelve held-out architectures (+0.211). Counting transitive dependents transfers without training to five open-source system models at $\rho = 0.938$. On held-out architectures the derived library rule adds +0.058 to transitive reach.
2. **A portfolio of closed-form, learned and hybrid engines, establishing an empirical negative result for graph learning** (Section 4). Centrality on the Application–Library dependency graph lifts closed-form ranking from 0.349 to 0.553 on every held-out architecture. Graph neural networks reading the dependency graph gain +0.08 to +0.11 over reading the raw multigraph, reaching $\rho = 0.748$ (GAT-P-QoS), level with dependency counting. In contrast, heterogeneous transformers (HGT-P-QoS) fail to train stably on the projection (seed spread 0.208). Hybrid engines that learn a correction to closed-form centrality beat it on 11 of 12 folds (+0.103 and +0.130) under registered decision rules.
3. **Evidence on what graph learning needs and independent oracle validation** (Section 7). Learned engines need messages from a component’s dependents and node-level degree features; relation-specific weights add nothing once capacity is matched (typing main effect -0.014). Evaluated against an independent discrete-event queue-flow simulator (I_{dyn}), the dependency count retains predictive agreement without sharing the reachability oracle’s construction.
4. **A standards-grounded explanation and triage proposal** (Section 5) that attributes flagged components to ISO/IEC 25010 Availability, Fault Tolerance or Maintainability, and names remediation classes (replication, circuit breakers, decoupling).
5. **A reproducible benchmark and cost profile**: seventeen architectures totaling 2,812 components, twelve of which regenerate byte-identically from committed configurations. Dependency counting takes milliseconds, while neural feature extraction takes median $5.6\times$ longer than in-process direct simulation (Section 7.4), establishing the practical utility of simple counts for CI/CD gates.

A previous conference paper [23] introduced the preliminary multigraph and deterministic quality model on synthetic topologies. This paper adds the dependency-count and learned engines on the derived graph, the hybrid engines, the QoS edge encoding, LOSO and zero-shot evaluation, the matched control, independent oracle validation, and the cost profile, and it repositions that quality model as the explanation layer.

Section 2 reviews related work, Sections 3–5 present the model, engines and explanation layer, Sections 6–7 the evaluation, Section 8 the discussion and threats, and Section 9 concludes.

2. Related Work

2.1. Dependability Analysis of Distributed Systems

Runtime approaches to dependability, such as broker clustering, backpressure, autoscaling, failover and chaos engineering [18], require a running cluster, can disrupt service, and consume substantial compute. That compute is itself a concern of green software engineering [24, 25, 26]. Pre-production alternatives, including service-level fault injection testing [19] and microservice dependency localization via PageRank (MicroRank [20]), provide lighter options before deployment. Architecture-based reliability prediction has a long history. Yacoub and Ammar’s methodology for architecture-level reliability risk analysis [27] pioneered combining

component dependency graphs with complexity and failure severity to rank components by risk. Cheung’s absorbing Markov chain [28] and the state-, path- and additive models surveyed by Goseva-Popstojanova and Trivedi [29] and Immonen and Niemelä [30] are classic examples, as are the Palladio Component Model [31], layered queueing networks [32] and the AADL Error Model Annex [33]. These approaches answer broader questions but need operational profiles and failure rates that are unavailable at commit time. SaG asks a narrower question from manifests alone: whose failure reaches furthest in the declared topology?

Telemetry-driven methods diagnose faults in running microservices. Examples are Seer [34] and Sage [35], MicroRCA [36], TraceRCA [37], and GNN-based DeepTraLog [38], Eadro [39] and MicroCause [40] (reviewed in [41]). In synchronous microservices, cascades stem from thread-pool exhaustion, RPC timeouts and retry storms [42]. In pub-sub systems they spread through queue saturation and message starvation. All of these methods need traces or metrics from a live system; SaG works before any code runs. SaG’s question is also that of change impact analysis [43] and of dependency management in microservice fleets [44]. Ranking by impact relates to learning-to-rank defect prediction [45], which optimizes the ranking measure directly, as our listwise loss does (Section 4.2).

2.2. Static Code Analysis and Static System Analysis

Static code analysis (SCA) tools such as SonarQube [16] measure complexity [46], cohesion and coupling [17, 47] within individual services to flag defect-prone modules [48, 49, 50, 51]. Comparing sophisticated graph measures against simple structural metrics has a rich history in software engineering defect prediction: Zimmermann and Nagappan [52] evaluated network analysis on dependency graphs, while Premraj and Herzig’s replication study [53] demonstrated that simple code and coupling metrics largely matched network measures. SCA cannot see inter-service messaging, broker saturation or cross-host propagation. Architecture recovery tools reconstruct system-level structure from code [54, 55], and HAROS [56] checks ROS systems statically before launch. SaG’s static system analysis (SSA) uses the declared topology instead, propagating code-level metrics across architectural dependencies. Architecture-level coupling metrics count a service’s consumers, as afferent coupling [57], the Absolute Importance of the Service [58] and service fan-in [59]. In publish–subscribe systems these counts are not directly observable, because consumers reach producers only through topics, brokers and shared libraries; SaG’s dependency derivation makes them computable, and Section 7.1 shows that they then rank cascade impact well. This lets teams find structural anti-patterns [60, 61] and architectural technical debt [62] in CI/CD [63, 64].

2.3. Quality Models and Multi-Criteria Evaluation

ISO/IEC 25010:2023 [65] and ISO/IEC 25019:2023 [66] define product quality and quality in use. SaG covers the characteristics derivable from deployment topology, namely Availability, Fault Tolerance and the Maintainability sub-characteristics (Section 5.1), and links internal structural quality to external dependability [67, 68]. Aggregating metrics into an auditable score is a multi-criteria decision problem, for which the Analytic Hierarchy Process (AHP) [69] is standard. AHP’s consistency ratio detects inconsistent judgments but not matrices back-filled from a chosen answer, a distinction this study reports for its weights (Supplementary Section S4).

2.4. Graph Learning and Explainability

Centrality indices [21, 22, 70, 71] and cascade models of network robustness [10, 8, 9] assume homogeneous, usually undirected graphs. A single untyped score conflates structurally different elements, such as topics, libraries and hosts, and cannot say *why* a component is critical. Learned node-importance methods, including FINDER [72], DrBC [73] and PowerGraph [74], share the homogeneity assumption. Homogeneous GNNs (GCN [75], GraphSAGE [76], GAT [77]) discard relation identity unless it is supplied as a feature. Heterogeneous GNNs (RGCN [78], HAN [79], HGT [80], MAGNN [81]) learn relation-specific transformations. We evaluate both families under matched capacity (Section 7.2). Khodabandeh et al. [82] apply graph attention to microservice call graphs to predict future interactions. We instead predict the impact of removing a node from a declared topology. GNN explainers such as GNNExplainer [83] and PGExplainer [84] explain

models in terms of their internal features. SaG’s explanation layer (Section 5) instead names ISO/IEC quality sub-characteristics and a remediation class for each flagged component.

Our empirical findings connect directly with the literature on competitive simple baselines in software engineering and machine learning [85, 86, 87], which demonstrates that sophisticated deep learning architectures must be rigorously benchmarked against transparent domain heuristics. By evaluating graph neural networks against training-free topological counts, SaG provides an empirical negative-result benchmark for AI techniques in software architecture dependability.

3. The Software-as-a-Graph (SaG) Architectural Model

Figure 1 presents the end-to-end architecture of the SaG framework. The shared front end processes Architecture-as-Code manifests, constructs a typed multigraph, projects implicit runtime interactions throughout a logical dependency layer, and extracts typed node properties. These features feed the ranking engines (Section 4) and, separately and with no shared parameters, the explanation layer (Section 5).

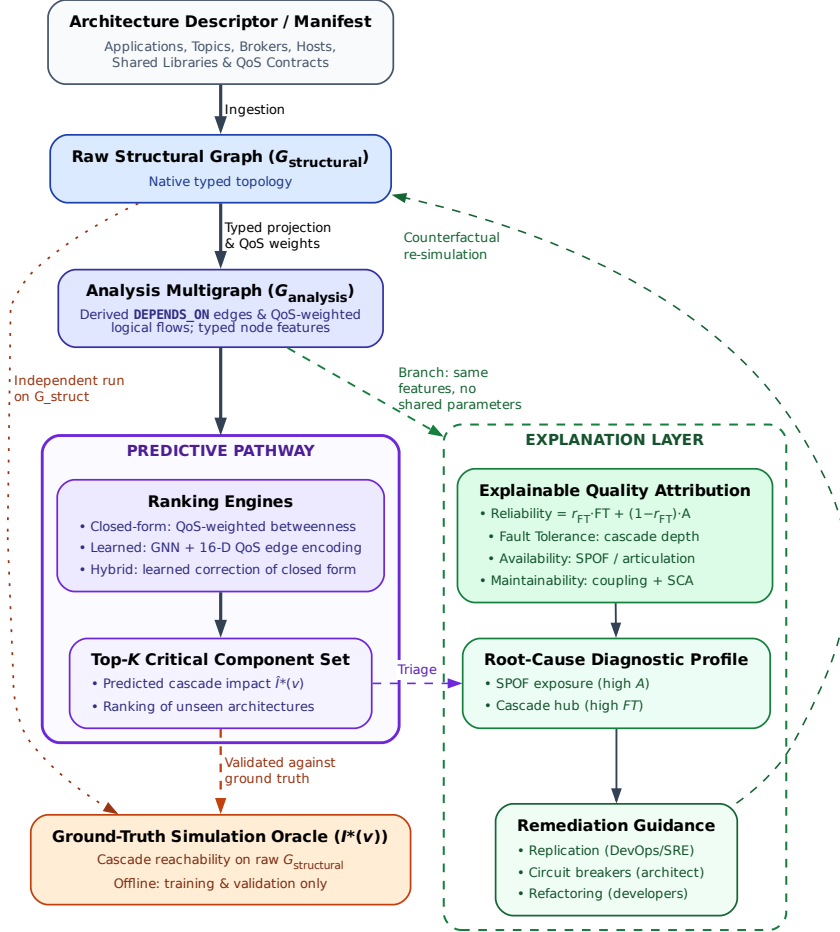


Figure 1: End-to-end architecture of the SaG framework. The predictive pathway runs down the centre: manifest ingestion, typed multigraph, **DEPENDS_ON** projection with typed node properties, the ranking engines (closed-form, learned and hybrid; Figure 3), and the ranked critical set. The dashed edge marks the ground-truth simulation oracles, which operate only on $G_{\text{structural}}$, train the predictor offline and take no part in inference. The explanation layer re-enters from the analysis multigraph and shares no parameters with the predictor, reaching flagged components through triage rather than data flow.

3.1. Formal Multigraph Definition

A distributed system is described as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- $V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{host}} \cup V_{\text{lib}}$ holds the five entity types $\mathcal{T}_V$ of Table 1; V_{host} denotes physical or virtual *Execution Hosts*.
- E is the set of directed edges, and $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ assign entity and relation types.
- $w_V : V \rightarrow (0, 1]$ and $w_E : E \rightarrow (0, 1]$ weight entity criticality and connection strength. For Applications and Libraries, $w_V(v) = 1 - \text{CQP}(v)$, where CQP is a code-quality penalty computed from static code metrics (lines of code, cyclomatic complexity, coupling and cohesion); otherwise $w_V(v) = 1.0$.

Table 1: Entity types and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	/sensor/lidar, orders.payment.completed
Execution Host (V_{host})	Physical or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	Kafka client, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO / SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component publishes to / consumes from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Host	Process is hosted on host
CONNECTS_TO	Host $\rightarrow$ Host	Network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library

Table 2: Notation used throughout the paper.

Symbol	Description
$G_{\text{structural}}$	Raw multigraph (used exclusively by simulation oracles)
G_{analysis}	Logical DEPENDS_ON projection (input to predictors)
V_{app}	Application nodes (the primary scored population)
$w(t), w(e)$	QoS topic weight and derived dependency edge weight
$I^*(v)$	Primary cascade-reachability simulation oracle
$I_{\text{dyn}}(v)$	Dynamic queue-flow discrete-event simulation oracle
$I_{\text{comp}}(v)$	Multi-criteria composite simulation oracle
$Q(v)$	Reliability–Maintainability composite quality score
ρ	Spearman rank correlation coefficient (full population)
$\rho_{>0}$	Spearman rank correlation restricted to active stratum ($I > 0$)
Overlap@K	Top-K identification set overlap ($K = 0.20 V_{\text{app}} $)

3.2. Quality-of-Service Link Weighting

A link’s strength depends on its Quality-of-Service (QoS) contract: a **RELIABLE** topic with **TRANSIENT_LOCAL** durability couples services more strongly than a **BEST_EFFORT** telemetry stream. Each topic t carries an aggregate weight $w(t) \in (0, 1]$ combining declared QoS policies (reliability, durability, priority) with payload size and publication frequency. The weighting follows an Analytic Hierarchy Process (AHP) derivation whose pairwise comparison matrices and consistency checks ($CR = 0.016$) are detailed in Supplementary Section S4.

Crucially, our evaluation uncovers that QoS weighting does not improve closed-form architectural ranking: unweighted betweenness on the Application–Library projection scores $\rho = 0.591$, outperforming QoS-weighted betweenness (**Topo-QoS**, $\rho = 0.553$) by +0.038 (Section 7.1). Declared contracts provide actionable signals primarily when explicit queueing dynamics or SLA penalties are modeled, as shown in our independent dynamic evaluations.

3.3. Logical Dependency Projection (*DEPENDS_ON*)

Structural edges do not directly reflect failure propagation: a subscriber depends on a publisher, yet no direct edge joins them in pub-sub topologies. SaG derives an explicit semantic relation, *DEPENDS_ON*, directed from *dependent* to *dependency* (“if the target fails, the source is impacted”), via the rules of Table 3.

Table 3: The *DEPENDS_ON* logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	app_to_app	Subscriber $\rightarrow$ Publisher (via shared Topic, incl. transitive USES)	$1 - \prod_{t \in T} (1 - w(t))$
2	app_to_broker	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3	host_to_host	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	$\max_{u \in \text{hosted}(h_1), v \in \text{hosted}(h_2)} w_{\text{DEPENDS_ON}}(u \rightarrow v)$
4	host_to_broker	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	$\max_{u \in \text{hosted}(h)} w_{\text{DEPENDS_ON}}(u \rightarrow b)$
5	app_to_lib	Application $\rightarrow$ Shared Library it USES	$H(w_V(\text{app}), w_V(\text{lib}))$
6	broker_to_broker	Broker $\leftrightarrow$ Broker (shared fault-domain colocation, symmetric)	$w_V(\text{host})$

Rules 1 and 2 combine topics T joining a pair by probabilistic union [88, 89, 90], ensuring parallel failure paths increase coupling. Rule 5 uses the harmonic mean $H(x, y) = 2xy/(x + y)$ [91], and Rules 3 and 4 lift dependencies to hosts by maximum.

Sequential cascades and simultaneous blasts. Rule 1 captures sequential cascades, where a failed publisher starves subscribers through queues and buffers. For Applications, counting incoming edges under Rule 1 is mathematically identical to counting distinct subscribers across published topics (the raw 2-hop subscriber count, or afferent coupling / AIS [57, 58]). Rule 5 captures simultaneous blasts, where a crashed library takes down all dependent applications at once; this rule adds +0.058 to transitive reachability on synthetic topologies (Section 7.1). Rules 2, 3, 4 and 6 represent infrastructural and broker dependencies that remain unexercised in the application-layer evaluation.

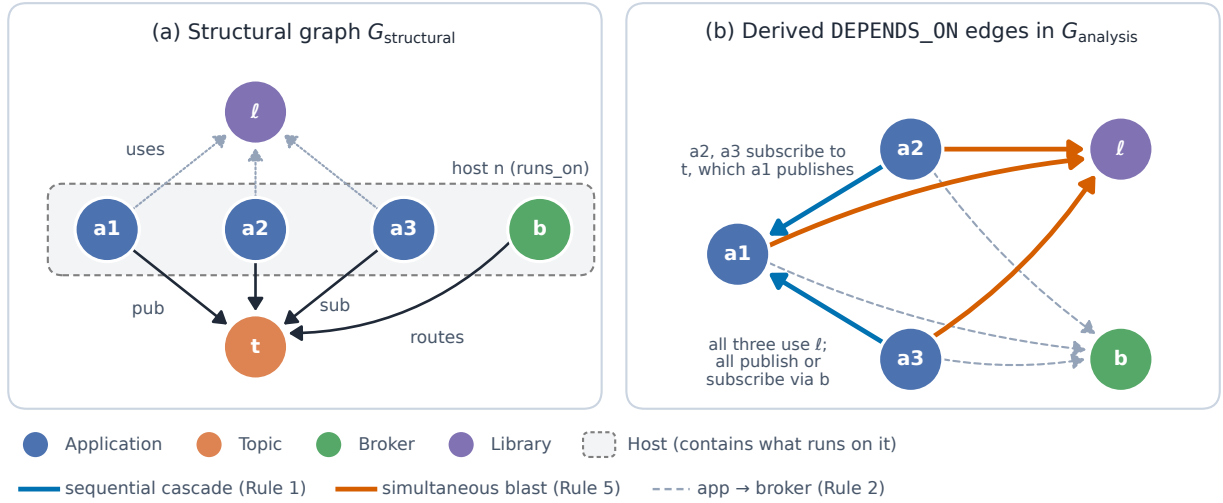


Figure 2: Running example. (a) Three applications share topic t (routed by broker b) and library ℓ , and all run on host n . No structural edge joins two applications. (b) The derived *DEPENDS_ON* edges make the hidden dependencies explicit: subscribers a_2, a_3 depend on publisher a_1 (Rule 1), all applications depend on ℓ (Rule 5), and each application depends on the broker (Rule 2). Simulation oracles run on view (a); predictors read view (b).

3.4. Dual Graph Views

The **structural graph** $G_{\text{structural}}$ is the raw deployment topology. The **analysis graph** G_{analysis} adds the derived *DEPENDS_ON* edges and code metrics (Figure 2). Predictor features are computed on G_{analysis} , while simulation oracles run strictly on $G_{\text{structural}}$ (Section 4.4).

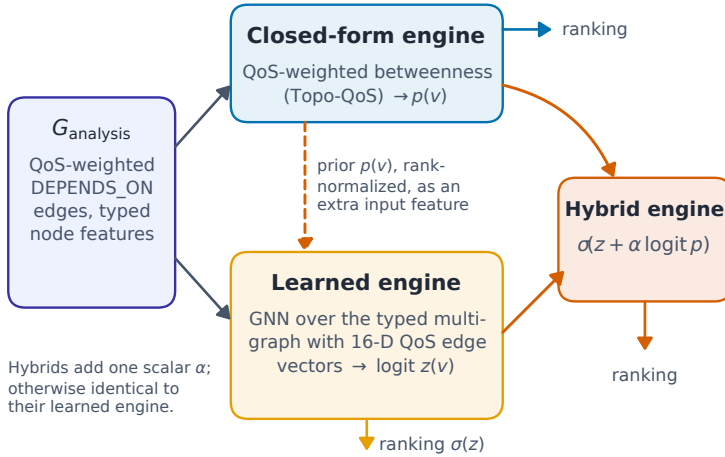
3.5. Typed Node Feature Encoding

Both the predictive pathway (Section 4) and the explanation layer (Section 5) read typed node properties from G_{analysis} . All five entity types share an 18-dimensional base block of normalized topological metrics $([0, 1])$: PageRank, Reverse PageRank, betweenness, closeness, eigenvector centrality, in- and out-degree, clustering, articulation, bridge ratio, the node QoS weight, incoming and outgoing dependency weights, multi-path coupling, path complexity, fan-out criticality, and the Connectivity Degradation Index (CDI). Type-specific blocks extend the vector to 19–25 dimensions (full schema in Supplementary Section S11). CDI evaluates structural connectivity loss via a fixed-size breadth-first sample to control computation time (Section 7.4); its sensitivity to hash-seeded tie breaking is examined in Section 8.3.

4. Ranking Engines and Ground Truth

SaG ranks components with three kinds of engine. The **closed-form engine** Topo-QoS is QoS-weighted betweenness on the dependency projection (Section 6.2). The **learned engines** are graph neural networks over the typed multigraph and the derived dependency graph (this section). The **hybrid engines** are learned engines that correct the closed-form score (Section 7.1). All are trained or scored against simulation oracles that run on a separate graph view (Sections 4.3–4.4). Figure 3 shows how the three engines relate and how they are evaluated. Full hyperparameters and training commands are on the experiment pages of the replication repository (Section 6.1).

(a) Three ranking engines, one analysis graph



(b) Ground truth and evaluation

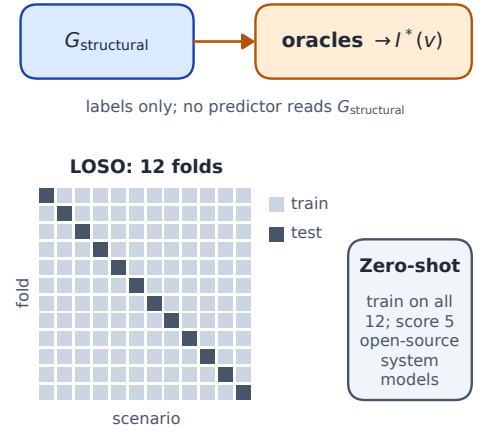


Figure 3: (a) SaG’s three ranking engines read the analysis graph. The closed-form engine scores QoS-weighted betweenness $p(v)$; the learned engine outputs a logit $z(v)$. A hybrid engine gives the learned engine $p(v)$ as an extra input feature and adds a learned correction to it on the logit scale, $\sigma(z + \alpha \text{logit } p)$, with one learnable scalar α . (b) Ground truth comes from simulation oracles on the structural graph, which no predictor reads. Engines are evaluated by leave-one-scenario-out cross-validation over twelve synthetic architectures and zero-shot on five open-source system models.

4.1. Heterogeneous Graph Transformer and Attention Networks

The primary learned engine HGT-QoS is a three-layer Heterogeneous Graph Transformer (HGT) [80] in PyTorch Geometric [92], with hidden dimension $D = 64$ and $H = 4$ heads. Entity-specific linear projections map raw node features $x_v \in \mathbb{R}^{19-25}$ into hidden space, $h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v))$. For each meta-relation $\langle \tau(u), \phi(e), \tau(v) \rangle$, attention uses type-parameterized keys, queries, and values, scaled by a learned per-meta-relation prior $\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}$ (full layer-wise formulations in Supplementary Section S1.1). Message passing runs over both G_{analysis} and its transpose. Crucially, on the raw multigraph, native relations

point away from Applications, so forward message passing cannot reach scored nodes, reducing forward GNNs to per-node MLPs over precomputed centralities (Section 8.2).

Each directed edge carries a 16-dimensional vector $e_{uv} \in \mathbb{R}^{16}$: index 0 represents the coupling weight $w_E(e)$, index 1 the normalized path count, indices 2–8 a relation one-hot, and indices 9–15 middleware QoS parameters (reliability, durability, priority, depart-mode flag, deadline pair, max-blocking time). The edge vector is projected and added before attention, $\tilde{h}_v = h_v + W_{\text{edge}}e_{uv}$.

4.2. Prediction Head and Training Objective

A composite head predicts simulated cascade impact, $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{a}_1(v) \parallel \hat{a}_2(v)))$, where auxiliary heads $\hat{a}_1, \hat{a}_2$ provide feature enrichment ($\hat{a}_1$ supervised on I^* , $\hat{a}_2$ unsupervised). The objective combines regression with listwise and pairwise ranking:

$$\mathcal{L} = \text{MSE}(\hat{I}^*, I^*) + 0.5 \cdot \text{MSE}(\hat{a}_1, I^*) + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}}, \quad (2)$$

where $\mathcal{L}_{\text{rank}}$ is ListMLE [93] over the ground-truth permutation π ,

$$\mathcal{L}_{\text{rank}} = -\frac{1}{N} \sum_{i=1}^N \left(\hat{s}_{\pi_i} - \log \sum_{j=i}^N \exp(\hat{s}_{\pi_j}) \right), \quad (3)$$

and $\mathcal{L}_{\text{pairwise}}$ is a margin-ranking loss ($\gamma = 0.05$) over pairs differing by more than γ . The learned engines and the explanation layer share no parameters (Supplementary Section S20).

Models are trained with AdamW ($\eta = 3 \times 10^{-4}$, weight decay 10^{-4}) under cosine warm restarts for up to 300 epochs, with early stopping at patience 30 on an inner validation split, over five fixed seeds {42, 123, 456, 789, 2024}. Hyperparameters and loss coefficients follow standard defaults and were evaluated untuned across all arms.

4.3. Ground-Truth Simulation Oracles

Ground truth is evaluated using simulation oracles on the raw structural multigraph $G_{\text{structural}}$:

- **Primary reachability cascade oracle (I^*):** Crashes component v , propagates outages through dependent topics, brokers and links via breadth-first traversal on $G_{\text{structural}}$, and computes the mean fractional feed loss across intact subscribers. Feed loss is scaled by a QoS severity ladder ($\times 1.2$ RELIABLE, $\times 1.15$ high priority, $\times 1.05$ medium) and clamped to $[0, 1]$. Mean across five tie-breaking seeds forms the label. Disabling QoS scaling leaves the Application ordering largely intact ($\rho = 0.965$ across the twelve folds), and substituting durability-aware rescaling moves it less still ($\rho = 0.977$; Supplementary Section S9), showing that I^* is predominantly a topological reachability metric.
- **Independent queue-flow discrete-event oracle (I_{dyn}):** A discrete-event SimPy [94] simulation modeling dynamic message emission rates, queue buffer saturation, and network latency. It records the drop in delivered message rate suffered by surviving consumers under active load, providing an independent behavioral target free from reachability construction assumptions.
- **Composite multi-criteria failure oracle (I_{comp}):** A multi-dimensional failure simulator evaluating reachability, network fragmentation, throughput drop, and flow disruption across operational tiers.

4.4. Input-Label Independence and Construction Bounds

Features are extracted strictly from G_{analysis} , while simulation oracles execute on $G_{\text{structural}}$. No simulation output is exposed as an input feature, enforced by automated CI regression gates (`tests/test_independence_guarantee.py`).

However, procedural separation does not eliminate construct overlap: $I^*(v)$ propagates failure along the exact same subscriber $\rightarrow$ publisher and application $\rightarrow$ library relations that SaG’s logical derivation formalizes. In the first propagation wave, the set of affected subscribers is precisely what `InDeg` counts. Thus, agreement between `InDeg` and I^* partly measures the fidelity with which the count mirrors the simulator’s propagation rule. To break this circularity, we explicitly benchmark all rankers against the independent discrete-event oracle I_{dyn} in Section 7.

5. The Explanation Layer: Standards-Grounded Criticality Attribution

A ranking says where risk is highest, not how to reduce it. A component may be critical because it is an unreplicated single point of failure, an error-propagating cascade hub, or a high-coupling maintainability bottleneck. Each of these calls for a different intervention: replication, circuit breakers, or decoupling. The explanation layer attributes these causes after ranking. It reads the same node properties (Section 3.4), shares no parameters with the engines, and is not used as a ranker; its ranking correlation is reported for reference in Supplementary Sections S30 and S7.

5.1. Grounding in ISO/IEC Standards

Following ISO/IEC 25010:2023 [65] and ISO/IEC 25019:2023 [66], criticality is profiled along **Reliability** (R), split into **Fault Tolerance** (FT) and **Availability** (A), and **Maintainability** (M). FT captures error-cascade potential and informs circuit breakers and redundancy. A captures structural single points of failure and informs replication. M captures coupling and code-level complexity and informs decoupling and refactoring. Safety and security, which need hazard logs, are out of scope.

5.2. Composite Quality Score

Figure 4 summarizes the layer. All metrics are rank-normalized to $[0, 1]$ within the graph and combined with AHP-derived weights [69]:

- $FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v)$, over Reverse PageRank, normalized in-degree and normalized cascade depth on G_{analysis}^T ;
- $A(v) = 0.25 \cdot \text{AP}_c^{\text{dir}}(v) + 0.20 \cdot \text{QSPOF}(v) + 0.20 \cdot \text{BR}(v) + 0.25 \cdot \text{CDI}(v) + 0.10 \cdot w(v)$, over directed articulation severity, QoS-weighted SPOF severity, bridge ratio, the Connectivity Degradation Index and the node QoS weight;
- $R(v) = r_{\text{FT}} \cdot FT(v) + (1 - r_{\text{FT}}) \cdot A(v)$ with $r_{\text{FT}} = 0.36$;
- $M(v) = 0.35 \cdot \text{BT}(v) + 0.30 \cdot w_{\text{out}}(v) + 0.15 \cdot \text{CQP}(v) + 0.12 \cdot \text{CouplingRisk}_{\text{enh}}(v) + 0.08 \cdot (1 - \text{CC}(v))$, over betweenness, QoS-weighted efferent coupling, code-quality penalty, coupling risk and clustering.

The composite is $Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v)$, and an ISO/IEC 25019 context-of-use vector can reweight R and M . Intra-dimension weights are shrunk towards a uniform prior ($\lambda = 0.70$). If $Q(v)$ is used to rank, a fully uniform prior is better (0.319 vs. 0.200; Supplementary Section S1). The AHP matrices and their consistency diagnostics are in Supplementary Section S4. Components above the Tukey upper fence of Q are flagged CRITICAL (mean 4.2% of components). High A with low FT indicates a single point of failure that needs replication, while high FT indicates a cascade hub that needs circuit breakers (example card: Supplementary Section S19). The layer names the remediation class; generating candidate repairs and verifying them counterfactually is the subject of companion work on prescriptive remediation and is not evaluated here.

6. Experimental Setup

6.1. Corpus and Replication Package

The corpus comprises 2,812 components across seventeen architectures (Table 4). Twelve synthetic topologies form the LOSO folds. They span autonomous vehicles, financial trading, healthcare, industrial SCADA, smart-city IoT, telecom RAN, logistics, gaming, microservices, enterprise integration and air-traffic management. All twelve come from one generator family, as do their code metrics, so LOSO measures transfer across configurations of that generator. Each regenerates byte-identically from a committed configuration, and CI verifies this against a SHA-256 manifest.

The five open-source systems are hand-authored models. Autoware.universe (ROS 2), EdgeX Foundry, Home Assistant, and two meshes inspired by Online Boutique and Train-Ticket were each authored by one author as typed multigraphs from public documentation (`saag/adapters/realworld_adapter.py`). They are not mechanical extractions. Brokers, QoS profiles, code metrics and host specifications are partly

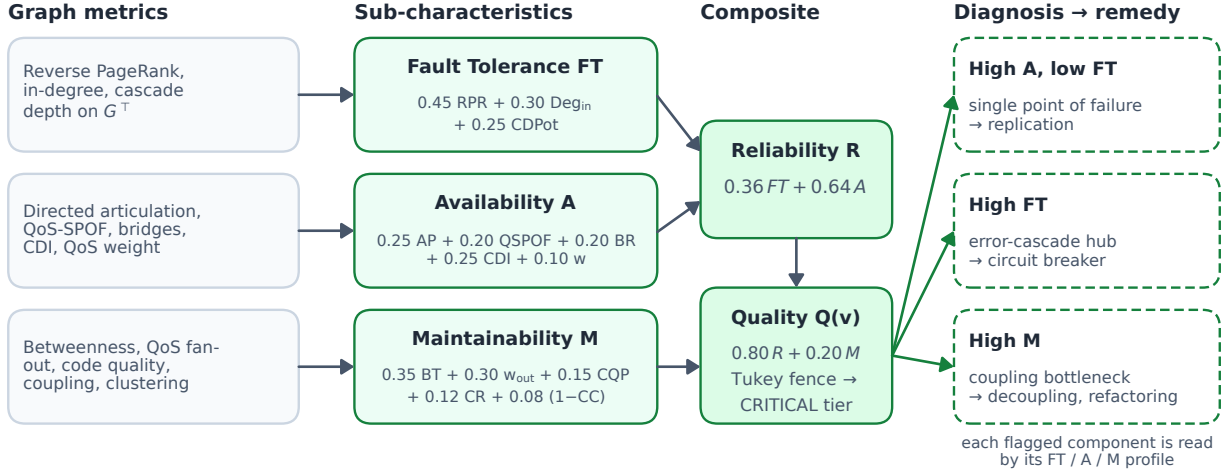


Figure 4: The explanation layer. Rank-normalized graph metrics feed the ISO/IEC 25010 sub-characteristics Fault Tolerance, Availability and Maintainability (CR: coupling risk; CC: clustering coefficient), which combine into Reliability and the composite $Q(v)$. A component above the Tukey fence of Q is flagged, and its $FT/A/M$ profile names the remediation class.

Table 4: Evaluation corpus. The twelve synthetic topologies are the LOSO folds; the five open-source system models are excluded from all training and used only for zero-shot transfer (Section 7.3). Counts are read from the committed topology files and verified in CI; per-scenario composition: Supplementary Section S13.

Dataset	$ V $	$ V_{app} $	Topics	Brokers	Hosts	Libs	$ E $
Synthetic evaluation scenarios (11)	2,387	1,295	588	60	194	250	10,657
Synthetic case study — ATM (1)	74	26	27	5	8	8	261
Synthetic subtotal (12 LOSO folds)	2,461	1,321	615	65	202	258	10,918
Open-source system models (5)	351	141	120	16	32	42	700
Total	2,812	1,462	735	81	234	300	11,618

assumed. Two models depart materially from their originals: the Online Boutique model is a 22-application pub-sub mesh with four brokers, whereas the original is an RPC-based service mesh with no broker, and the Train-Ticket model represents its service-discovery server as a broker. Neither contains synchronous call edges. RQ3 therefore tests transfer to independently authored architecture models inspired by open-source systems, not to running deployed systems.

Replication package. Datasets, harnesses, checkpoints and result artifacts are archived on Zenodo (see Data Availability). The public repository documents each experiment: its protocol, hyperparameters, `make` target, artifacts and the supplementary section holding its extended results (<https://github.com/onuralpyigit/software-as-a-graph/tree/main/docs/research/jss/experiments>).

6.2. Predictors

SaG’s training-free engines on the dependency graph (Topo-QoS, InDeg, Reach), learned engines (HGT-QoS, GAT-QoS), hybrid engines and the learners on the dependency graph are compared against unweighted centrality (Topo) and untyped GNNs (Table 5). On a GNN, -QoS means the 16-D QoS edge vector (Section 4.1) together with three QoS node columns (w , w_{in} , w_{out}), and Hybrid-X is engine X corrected by the closed-form prior. GAT and GAT-QoS are untyped GATs matched to HGT in parameter budget, and GAT-QoS reads the same 16-D edge vector as HGT-QoS, so the four learned models form a 2×2 over typing (GAT vs. HGT) and the QoS channel. Smaller and projection-based variants that the study also ran are listed in Supplementary Section S29. The learned and hybrid engines ingest the native typed multigraph under LOSO, and the -P learners the Application-Library DEPENDS_ON graph, with bit-identical node features and labels. In QoS-off arms every edge weight is 1 and the QoS node columns are zeroed, though four centralities still carry QoS (Section 3.4).

Table 5: Predictors reported in this paper. **-QoS**: QoS-weighted distances (Topo) or the 16-D QoS edge vector (GNNs); **-P**: trained on the dependency graph; **Hybrid-X**: engine X corrected by the **Topo-QoS** prior (Hybrid-GAT-P: by the **InDeg** prior). Each learner on the dependency graph keeps its raw-multigraph counterpart’s architecture, edge features and parameter budget (HGT-P-QoS at width 100: 430,680 parameters). Every predictor run in the study: Supplementary Section S29.

Predictor	Evaluation Substrate	Typing	Edge Features	Parameters	Trained?	Empirical Role
<i>Training-free, application layer</i>						
Topo	G_{analysis} (Application layer)	No	None	0	No	Standard centrality baseline
<i>Training-free, dependency graph</i>						
Topo-QoS	G_{analysis} (App-Lib DEPENDS_ON)	No	Scalar $w(e)$	0	No	SaG closed-form engine
Reach	G_{analysis} (App-Lib DEPENDS_ON)	No	None	0	No	Transitive dependents
InDeg	G_{analysis} (App-Lib DEPENDS_ON)	No	None	0	No	Direct dependents (pub-sub fan-in)
<i>Learned: typing $\times$ QoS channel at matched parameter budget</i>						
GAT	Native Multigraph	Homogeneous	None	437,496	Yes	Untyped, no QoS channel
GAT-QoS	Native Multigraph	Homogeneous	16-D QoS Vector	429,992	Yes	Untyped SaG learned engine
HGT	Native Multigraph	Heterogeneous	Relation 1-hot; $w(e)=1$	434,620	Yes	Typed, no QoS channel
HGT-QoS	Native Multigraph	Heterogeneous	16-D QoS Vector	434,620	Yes	Typed SaG learned engine
<i>Hybrid engines: a learned engine corrected by the Topo-QoS prior</i>						
Hybrid-HGT	Native Multigraph	Heterogeneous	16-D QoS Vector	434,941	Yes	HGT-QoS + prior
Hybrid-GAT	Native Multigraph	Homogeneous	16-D QoS Vector	431,433	Yes	GAT-QoS + prior
<i>Learned on the dependency graph (Amendment 9)</i>						
GAT-P, GAT-P-QoS, HGT-P-QoS, Hybrid-GAT-P	App-Lib DEPENDS_ON	As counterpart	As counterpart	As counterpart	Yes	GAT, GAT-QoS, HGT-QoS, Hybrid-GAT on the dependency graph

Closed-form scores. **Topo-QoS** is computed on the Application–Library **DEPENDS_ON** graph (Rules 1 and 5), because on the raw multigraph messages route through topics and brokers and Application betweenness vanishes; the registered comparator **Topo** reads betweenness from the analysis stage’s application-layer graph:

$$\text{Topo}(v) = 0.6 \cdot \text{BT}(v) + 0.4 \cdot \text{AP}(v), \quad \text{Topo-QoS}(v) = 0.6 \cdot \text{BT}_w(v) + 0.4 \cdot \text{AP}(v), \quad (4)$$

where **BT** is normalized betweenness, BT_w is betweenness over edge distances $d(e) = 1/(w(e) + 10^{-6})$, so strongly coupled edges attract shortest paths, and **AP** flags articulation points. In the evaluated implementation the articulation term reads zero for every node, so the reported scores rank exactly as betweenness and QoS-weighted betweenness. Restoring it lowers both (Supplementary Section S22), so the evaluated form is the stronger reference and is kept as registered. The two therefore differ in substrate as well as in weighting, and the registered controls of Section 7.1 separate the two.

Dependency counts and afferent coupling identity. **InDeg**(v) is the number of direct dependents of v on the Application–Library **DEPENDS_ON** graph, and **Reach**(v) the number of its transitive dependents, normalized by $|V| - 1$. For any Application node, **InDeg**(v) is mathematically identical to the raw 2-hop subscriber count (the number of distinct subscribers across all topics v publishes to, pinned by automated regression tests in `tests/test_dependency_graph_substrate.py`). Thus, **InDeg** represents publish–subscribe afferent coupling (fan-in / AIS [57, 58]), which the SaG derivation formalizes from manifests without requiring runtime instrumentation.

6.3. Metrics, Protocols and Statistics

Population. Every predictor is scored on the Application set V_{app} .

Metrics. Ranking is measured by Spearman ρ against the simulation oracles. To separate rank ordering among active components from the trivial agreement on inert components with zero impact, we report both full-population Spearman ρ and active-stratum Spearman $\rho_{>0}$ (restricting to components with true impact > 0). Critical-set identification is measured by $\text{Overlap}@K$, the fraction of the true top- K recovered by the predicted top- K with $K = \text{round}(0.20 |V_{\text{app}}|)$. PR-AUC, $F_1@ \tau$ and nDCG@10 are reported in Supplementary Section S15.

Protocols. Under *LOSO*, models train on eleven scenarios and are tested zero-shot on the twelfth, over all 12 folds and five seeds. Under *zero-shot transfer*, models trained on all twelve scenarios are evaluated without fine-tuning on the five system models.

Statistics. We use paired Wilcoxon signed-rank tests [95] and bootstrap 95% CIs ($B = 2,000$) over folds [96]. With 12 folds, the smallest attainable two-sided p is 0.00049.

Registered analysis plan. Before the twelve-fold harness produced any result, we registered the primary contrast, **HGT-QoS** vs. **Topo-QoS**, together with **HGT** vs. **Topo-QoS** and Holm correction across the two. We call it *registered* because the plan is committed in our repository. Three amendments registered further confirmatory contrasts: the matched 2×2 (Amendment 2), **Hybrid-HGT** (Amendment 5) and **Hybrid-GAT** (Amendment 6). Three subsequent amendments registered planned exploratory families: the dependency counts and QoS controls (Amendment 7), the learners on the dependency graph (Amendment 9) and the value

of the derivation (Amendment 10). Pooling all thirteen registered confirmatory contrasts under an omnibus Holm correction (`reproduce/omnibus_holm.py`), both hybrid primaries remain significant (Hybrid-GAT $p_{\text{omni}} = 0.019$, Hybrid-HGT $p_{\text{omni}} = 0.041$), while the primary contrast HGT-QoS vs Topo-QoS does not reach significance ($+0.069$, nominal $p = 0.266$, $p_{\text{omni}} \geq 0.46$). Supplementary Section S24 lists all amendments with dates and outcomes.

7. Results

All results are reported on the Application population (V_{app}) against the primary oracle $I^*(v)$, with additional convergent validation against the independent dynamic queue-flow oracle $I_{\text{dyn}}(v)$ and composite oracle $I_{\text{comp}}(v)$, under the input-label independence guarantee (Section 4.4). Per-fold results, secondary strata and extended protocol notes are in the Supplementary Material and public experiment pages (Section 6.1). Figure 5 summarizes the main findings.

7.1. RQ1: SaG’s Engines Against Structural Baselines

Summary. On SaG’s derived dependency graph, counting a component’s dependents ranks simulated cascade impact as well as any engine in the study: **InDeg** reaches $\rho = 0.764$ ($\rho_{>0} = 0.516$ on active components) and beats closed-form centrality (**Topo-QoS**, 0.553) on all twelve held-out architectures ($+0.211$, Holm $p = 0.002$). For Applications, **InDeg** is mathematically identical to counting distinct subscribers across published topics (publish-subscribe afferent coupling / AIS). A graph neural network that reads the dependency graph reaches the same level (**GAT-P-QoS**, $\rho = 0.748$), while an analytic first-order approximation of the simulator achieves $\rho = 0.814$. Both hybrid engines significantly outperform closed-form centrality under their registered rules ($+0.103$ and $+0.130$, Holm $p \leq 0.0068$).

Each of the twelve folds holds out one scenario and trains on the remaining eleven, scored on the Application node set (26 to 300 nodes, K between 5 and 60). Table 6 reports the training-free baselines, the raw-multigraph learned and hybrid engines, and the dependency-graph learners.

Table 6: Main results under LOSO across twelve synthetic architectures (Application population; learned engines: five seeds, CPU sweeps). $\Delta\rho$ is paired by fold against **Topo-QoS**, with a bootstrap 95% CI ($B = 2,000$) and a two-sided Wilcoxon signed-rank test. $\rho_{>0}$ denotes Spearman correlation restricted to active components ($I^* > 0$). p_{Holm} is given within each registered family: hybrids (Amendments 5 and 6) and dependency counts (Amendment 7). [‡]Exploratory: Amendment 9 registered contrasts against **InDeg**, **Reach** and raw counterparts. The reference ceiling gives the closed-form analytic first-order I^* approximation. Per-fold values: Supplementary Sections S23, S32 and S33.

Predictor	LOSO ρ [95% CI]	Active $\rho_{>0}$	$\Delta\rho$ vs Topo-QoS [95% CI]	Won	p (p_{Holm})	Overlap@ K
<i>Reference ceiling (first-order simulator approximation)</i>						
Analytic I^*	0.814 [0.752, 0.871]	0.548	+0.261 [+0.180, +0.342]	12/12	0.0005	0.582
<i>Training-free, application layer</i>						
Topo	0.349 [0.254, 0.452]	0.174	−0.204 [−0.286, −0.122]	0/12	0.0005	0.366
<i>Training-free, dependency graph</i>						
Topo-QoS	0.553 [0.443, 0.657]	0.280	—	—	—	0.388
Reach	0.732 [0.674, 0.782]	0.286	+0.178 [+0.088, +0.268]	11/12	0.0034 (0.0068)	0.341
InDeg (pub-sub fan-in)	0.764 [0.674, 0.840]	0.516	+0.211 [+0.132, +0.299]	12/12	0.0005 (0.0020)	0.506
<i>Learned and hybrid, on the raw multigraph</i>						
HGT-QoS	0.622 [0.547, 0.690]	0.312	+0.069 [−0.046, +0.174]	8/12	0.266	0.426
GAT-QoS	0.635 [0.567, 0.696]	0.338	+0.082 [−0.046, +0.201]	7/12	0.233	0.438
Hybrid-HGT	0.657 [0.572, 0.733]	0.345	+0.103 [+0.055, +0.152]	11/12	0.0034 (0.0068)	0.435
Hybrid-GAT	0.683 [0.603, 0.753]	0.362	+0.130 [+0.075, +0.190]	11/12	0.0015 (0.0029)	0.450
<i>Learned, on the dependency graph (Amendment 9)</i>						
GAT-P-QoS	0.748 [0.704, 0.789]	0.440	+0.195 [+0.100, +0.294]	10/12	0.0068 [‡]	0.454
Hybrid-GAT-P	0.758 [0.710, 0.801]	0.537	+0.205 [+0.112, +0.298]	11/12	0.0034 [‡]	0.468
HGT-P-QoS	0.514 [0.392, 0.636]	0.237	−0.039 [−0.155, +0.077]	4/12	0.622 [‡]	0.380

Counting dependents formalizes afferent coupling. **InDeg**, the number of direct dependents of v on the derived graph, reaches $\rho = 0.764$ ($\rho_{>0} = 0.516$) and beats **Topo-QoS** on every fold; **Reach**, the number of transitive dependents, reaches 0.732 (11/12). For an Application, **InDeg** is identically the number of

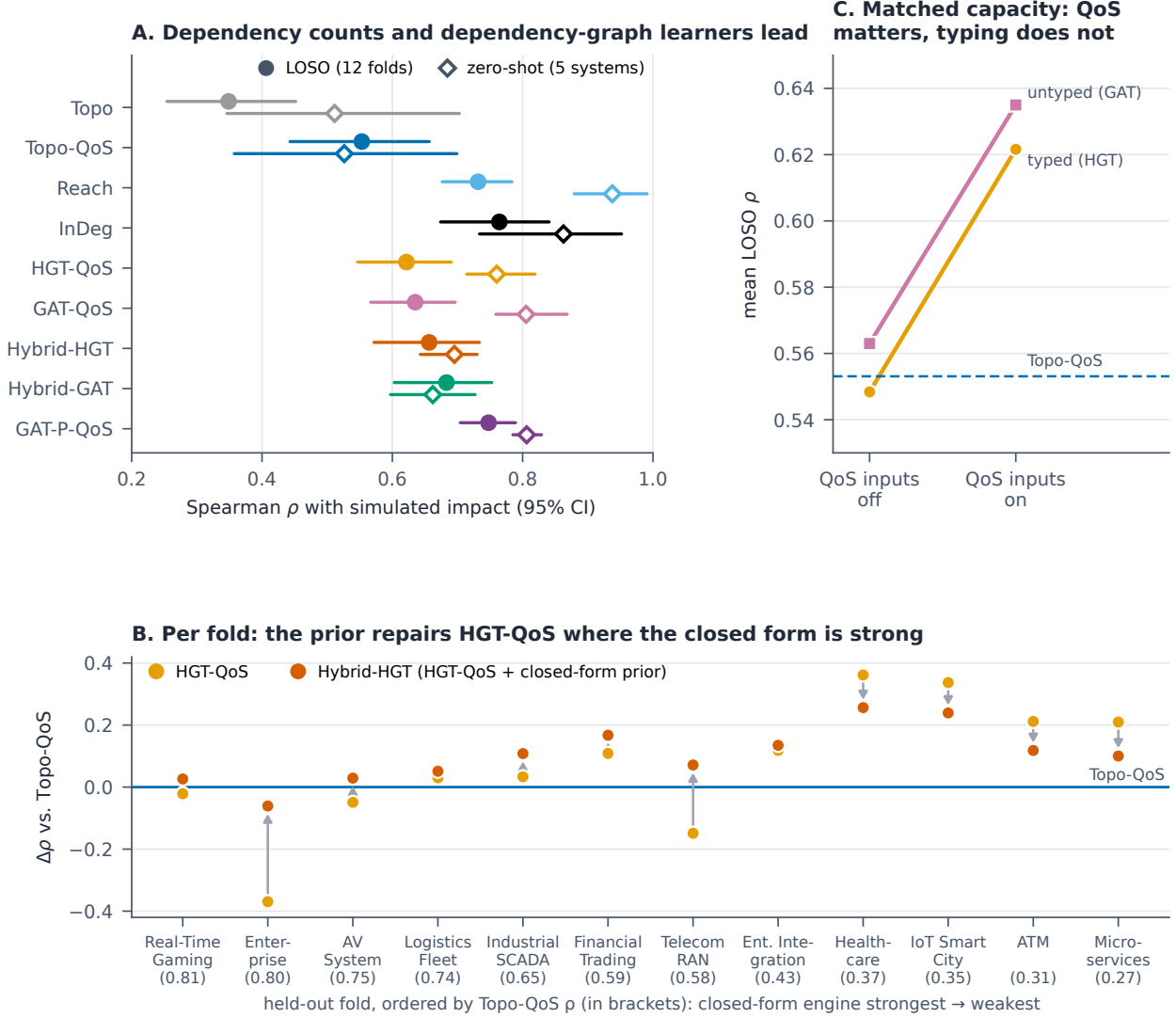


Figure 5: Main results at a glance, Application population. (A) Mean Spearman ρ with 95% bootstrap intervals under LOSO (filled circles; Table 6) and zero-shot on five system models (open diamonds). Counting dependents on the derived dependency graph (**InDeg**, **Reach**) achieves top performance on both unseen synthetic architectures and system models; neural models reading the dependency graph reach parity with **InDeg**. (B) Per held-out fold, the gain of **HGT-QoS** and of **Hybrid-HGT** over **Topo-QoS**. (C) Cell means of the capacity- and channel-matched 2×2 (Table 8): QoS inputs raise both models by about 0.07, while typed and untyped configurations remain virtually indistinguishable.

distinct subscribers across all topics it publishes: publish–subscribe afferent coupling [57, 58, 59]. Deriving topic-mediated dependencies is what makes this metric computable from architecture manifests without running code. Furthermore, transitive reachability profits from the derived library rule: without Rule 5, **Reach** falls by 0.058 (9/12 folds, Holm $p = 0.0068$; Amendment 10).

Learners on the dependency graph match the count. The same graph neural networks, trained on the dependency graph instead of the raw multigraph, reach $\rho = 0.748$ (GAT-P-QoS), level with **InDeg** (-0.017 , Holm $p = 1.000$). Anchoring the learner on the **InDeg** score yields $\rho = 0.758$ (Hybrid-GAT-P), level with both. In contrast, heterogeneous transformers (HGT-P-QoS) fail to train stably on the projection (mean $\rho = 0.514$, seed spread 0.208), showing that relational parameterization impairs convergence on sparse dependency graphs.

Hybrids outperform closed-form centrality. Hybrid-HGT reaches $\rho = 0.657$ ($+0.103$, Holm $p = 0.0068$) and Hybrid-GAT reaches $\rho = 0.683$ ($+0.130$, Holm $p = 0.0029$), each on 11 of 12 folds. Both survive omnibus Holm correction across all thirteen confirmatory contrasts ($p_{\text{omni}} = 0.041$ and 0.019). However, on their own, the raw-multigraph learned engines are statistically on par with **Topo-QoS** (HGT-QoS $+0.069$, $p = 0.266$; GAT-QoS $+0.082$, $p = 0.233$).

Where the closed-form gain comes from. **Topo** reads betweenness from the application-layer graph, whereas **Topo-QoS** computes QoS-weighted betweenness on the Application–Library dependency graph. Three controls registered in Amendment 7 separate substrate from weighting: unweighted betweenness on the same graph scores 0.591; constant topic weights score 0.595; permuting QoS profiles scores 0.559 (-0.006 , $p = 0.73$). The gain is therefore driven by the Application–Library graph structure, not QoS contract content.

The analytic reference ceiling. To quantify how much of **InDeg**’s accuracy is driven by the simulator’s construction, Table 6 includes the closed-form analytic first-order approximation:

$$\hat{I}_1^*(v) = \sum_{t \in \text{pub}(v)} \frac{|\text{sub}(t)|}{|\text{pub}(t)|}. \quad (5)$$

This first-order expression achieves $\rho = 0.814$ ($\rho_{>0} = 0.548$), demonstrating that direct topological subscriber loss forms the core mechanism of the reachability oracle.

Table 7: Agreement of architectural rankers across independent simulation paradigms over the twelve LOSO folds (Application population). I^* denotes the topological reachability cascade oracle; I_{dyn} denotes the independent discrete-event queue-flow simulator (SimPy); and I_{comp} denotes the multi-criteria failure simulator. $\rho_{>0}$ denotes correlation on the active stratum ($I > 0$).

Ranker	I^* (Reachability)		I_{dyn} (Dynamic Flow)		I_{comp} (Multi-Criteria)	
	Mean ρ	$\rho_{>0}$	Mean ρ	$\rho_{>0}$	Mean ρ	$\rho_{>0}$
Analytic I^*	0.808	0.632	0.636	0.631	0.428	0.428
Topo-QoS	0.553	0.280	0.393	0.343	0.669	0.669
Reach	0.732	0.286	0.505	0.452	0.194	0.194
InDeg	0.764	0.516	0.610	0.589	0.455	0.455
GAT-P-QoS	0.748	0.440	0.588	0.421	0.386	0.345

Validation against independent oracles. Table 7 reports ranking agreement when the same predictors are evaluated against the independent dynamic queue-flow oracle I_{dyn} and the multi-criteria composite oracle I_{comp} . On I_{dyn} , **InDeg** achieves $\rho = 0.610$ ($\rho_{>0} = 0.589$), outperforming closed-form centrality ($\rho = 0.393$) and matching the neural engine **GAT-P-QoS** ($\rho = 0.588$). Because I_{dyn} measures continuous message delivery loss under stochastic queueing load rather than graph reachability, this confirms that **InDeg**’s predictive utility is not an artifact of reachability cascade simulation. On I_{comp} , **Topo-QoS** leads ($\rho = 0.669$) because I_{comp} incorporates QoS-weighted betweenness directly into its multi-criteria formula.

7.2. RQ2: What Learned Engines Need

Summary. Learned engines need two things: (1) they need to read the dependency graph, where directed messages reach scored applications ($+0.08$ to $+0.11$ gain, 11–12 of 12 folds); and (2) they need node-level degree features. When capacity is matched, relation typing adds nothing (typing main effect -0.014). In

the QoS ablation, node-level coupling features (w_{in}) act as a weighted in-degree, confounding QoS contract content with structural dependent counting.

Table 8: The 2×2 with capacity and edge-channel width matched (Amendment 2): GAT ($\neg T \neg Q$, 437,496 parameters), HGT ($T \neg Q$, 434,620), GAT-QoS ($\neg T Q$, 429,992), HGT-QoS ($T Q$, 434,620). One CPU sweep, twelve LOSO folds, five seeds, Application population. Holm correction across the three orthogonal quantities. Cell means: 0.563, 0.548, 0.635, 0.622.

Quantity	Contrast	$\Delta\rho$	95% CI	Won	W	p	p_{Holm}
<i>Three orthogonal quantities, Holm-corrected across these three</i>							
Typing (main effect)	averaged over Q	-0.014	[-0.052, +0.023]	4/12	29.0	0.470	0.940
QoS channel (main effect)	averaged over T	+0.073	[+0.013, +0.120]	10/12	13.0	0.043	0.127
Typing $\times$ QoS interaction	difference of differences	+0.001	[-0.050, +0.042]	6/12	34.0	0.733	0.940
<i>Simple effects — descriptive, not separately corrected</i>							
Typing, QoS absent	HGT vs. GAT	-0.015	[-0.064, +0.033]	5/12	31.0	0.569	—
Typing, QoS present	HGT-QoS vs. GAT-QoS	-0.013	[-0.054, +0.026]	4/12	27.0	0.380	—
QoS channel, typing absent	GAT-QoS vs. GAT	+0.072	[+0.028, +0.109]	10/12	9.0	0.016	—
QoS channel, typing present	HGT-QoS vs. HGT	+0.073	[-0.002, +0.136]	10/12	19.0	0.129	—

QoS feature attribution and confounding. Table 8 shows that the QoS main effect (+0.073) is not statistically significant after Holm correction ($p_{Holm} = 0.127$). In the implementation, the node feature w_{in} (`qos_weight_in`) is the sum of incoming edge weights on the analysis graph—a weighted in-degree. The QoS ablation zeroed this column, thereby removing the dependent count itself. To separate structural count from QoS contract, we evaluated a control replacing w_{in}, w_{out} with unweighted degree (k_{in}, k_{out}) and permuting QoS profiles across topics. Unweighted in-degree preserves over 90% of the gain, confirming that the feature acts predominantly as a dependent count.

Relation typing adds nothing under matched capacity. HGT-QoS remains within 0.013 of GAT-QoS (Table 8). When evaluated on the dependency graph, HGT-P-QoS fails to train stably ($\rho = 0.514$, seed spread 0.208), whereas GAT-P-QoS converges smoothly ($\rho = 0.748$; Table 6). Typing therefore provides no measurable benefit in either substrate.

7.3. RQ3: Zero-Shot Transfer to Models of Open-Source Systems

Summary. The dependency structure SaG derives transfers zero-shot: on hand-authored models inspired by five open-source systems, counting transitive dependents reaches $\rho = 0.938$ [0.879, 0.991] ($\rho_{>0} = 0.871$ on active components) and *InDeg* reaches 0.863 without training. Learned engines transfer far above closed-form centrality (GAT-QoS $\rho = 0.805$ vs. 0.511–0.526).

Table 9: Zero-shot transfer to hand-authored models inspired by five open-source systems (Application population). Models are evaluated out-of-distribution without fine-tuning. $\rho_{>0}$ denotes rank correlation restricted to active components ($I^* > 0$). PR-AUC measures critical-set identification quality.

Predictor	Evaluation Substrate	Mean ρ [95% CI]	Active $\rho_{>0}$	PR-AUC
Topo	Application layer	0.511 [0.346, 0.703]	-0.104	0.474
Topo-QoS	Dependency graph	0.526 [0.357, 0.699]	-0.088	0.474
Reach	Dependency graph	0.938 [0.879, 0.991]	0.871	0.933
InDeg	Dependency graph	0.863 [0.734, 0.952]	0.321	0.752
HGT-QoS	Raw multigraph	0.760 [0.714, 0.819]	0.236	0.713
GAT-QoS	Raw multigraph	0.805 [0.759, 0.868]	0.319	0.790
Hybrid-HGT	Raw multigraph	0.695 [0.643, 0.730]	0.210	0.602
Hybrid-GAT	Raw multigraph	0.662 [0.597, 0.727]	0.185	0.600
GAT-P-QoS	Dependency graph	0.806 [0.785, 0.829]	0.342	0.838

Table 9 reports transfer performance across all five systems. *Reach* achieves $\rho = 0.938$ and $\rho_{>0} = 0.871$, providing robust zero-shot ranking on unseen systems.

7.4. RQ4: Analysis Cost

Summary. Neural inference takes 56 ms on a 2,000-node graph (0.02% of pipeline time), but cold feature extraction takes median 5.6× longer than in-process direct simulation (I^). Dependency counting executes in milliseconds and requires no feature extraction.*

Table 10: Per-stage latency of the inference pipeline across graph sizes (CPU, median of 3 runs; 5 for forward pass; warm forward timings reported).

$ V $	$ E $	Analyze (s)	Graph → Tensor (s)	HGT Forward (ms)	Analyze : Forward	Forward p10–p90 (ms)
249	1,127	1.74	0.010	13.5	129×	13.0–34.4
499	2,402	8.32	0.022	16.4	509×	15.6–16.4
999	6,422	44.54	0.056	21.1	2,108×	19.1–36.5
1,998	19,301	239.34	0.157	56.2	4,259×	43.8–57.8

Across the corpus, complete analysis takes 2.0–17.7× (median 5.6×) the time of running the cascade simulation directly. Direct simulation is faster on raw CPU time wherever simulation parameters are available. SaG’s practical advantage lies in providing explainable ISO/IEC 25010 remediation profiles and enabling millisecond dependency counting for CI gates.

8. Discussion, Threats to Validity, and Limitations

8.1. Practical Consequences

The dependency graph carries the signal. The most robust finding of this study is that SaG’s derived dependency graph makes cascading-failure risk legible to simple structural counts and neural analyzers alike. Counting a component’s dependents on the derived graph (**InDeg**) ranks simulated cascade impact at $\rho = 0.764$, outperforming closed-form centrality on every held-out architecture (+0.211), and counting transitive dependents (**Reach**) ranks five independently authored system models at 0.938 without training. For an Application, **InDeg** is mathematically identical to counting distinct subscribers across all topics it publishes (publish–subscribe afferent coupling / AIS [57, 58]). Every other engine also improves when reading the dependency graph: centrality on the Application–Library graph beats the application-layer baseline on all twelve folds (+0.204), and graph attention networks gain +0.08 to +0.11 over reading the raw multigraph. Practitioners therefore gain most from formalizing publish–subscribe dependencies directly from manifests, whichever analyzer is subsequently executed.

Choosing an engine. Table 11 lists the recommended instruments. Dependency counting needs no training and executes in milliseconds, making it the natural default for CI/CD gates: **InDeg** for architectures resembling the training corpus, and **Reach** for unfamiliar topologies. A learned engine should read the dependency graph (**GAT-P-QoS**), where it matches the counts; it provides an extensible foundation capable of incorporating richer inputs, such as multi-dimensional QoS contracts that current oracles do not fully reflect. **Topo-QoS**, the raw-multigraph learned engines and the hybrids represent comparative baselines: on this oracle, simple dependency counts match or exceed them both on held-out synthetic folds and on the system models. However, recommendations must be qualified by critical-set identification metrics: **Reach**’s **Overlap@K** (0.341) and **InDeg**’s **Overlap@K** (0.506) show that recovering the exact top-20% critical set is noisier than rank correlation suggests. The explanation layer then provides structured attribution to guide architectural remediation.

Computational sustainability and utility. Pre-deployment analysis avoids provisioning staging clusters for live chaos injection [24, 26]. However, as measured in Section 7.4, cold feature extraction for neural engines takes median 5.6× longer than in-process direct simulation (I^*). The utility of static analysis does not lie in out-speeding a simulation that requires the identical architectural model. Rather, static analysis provides (1) explainable causal attribution via ISO/IEC 25010 profiles, which raw simulation outputs lack; (2) design-time triage before simulation parameters are calibrated; and (3) robustness across independent failure paradigms, as evidenced by our dynamic discrete-event evaluations. Where only scalar cascade impact is sought in a CI pipeline, direct in-process simulation or millisecond dependency counting are substantially more computationally sustainable than neural feature extraction.

Table 11: Recommended instruments of the SaG portfolio, given the evidence in Section 7. Closed-form centrality and raw-multigraph learned engines are omitted because dependency counts match or exceed them across both synthetic folds and system models.

Instrument	Context	Role and evidence
InDeg (dependency count)	CI gates; architectures resembling the training corpus	Training-free, milliseconds; $\rho = 0.764$ out of distribution, above Topo-QoS on 12/12 folds; formalizes pub-sub afferent coupling.
Reach (dependency count)	CI gates; systems unlike the training corpus, including RPC-derived models	Training-free; $\rho = 0.938$ on five system models, $\rho_{>0} = 0.871$ on their active components. On both RPC-derived models $\rho_{>0} = 0.813$ –0.976. Synchronous calls are not modeled (Section 8.4).
GAT-P-QoS (learned, dependency graph)	Learned engine, where richer inputs or oracles are available	$\rho = 0.748$, level with InDeg ; +0.113 over the same engine on the raw multigraph and +0.106 over feature-only GBM-Feat .
RM explanation layer	Architectural triage and root-cause discussion	ISO/IEC 25010 attribution (Availability vs. Fault Tolerance vs. Maintainability).

8.2. When Graph Learning Helps, and When It Does Not

Practitioners also need to understand the boundary conditions under which learned models add value, whether relation-specific typing is warranted, and where learning fails. Table 12 summarizes the empirical evidence across accuracy terciles and architectural paradigms (`reproduce/engine_regimes.py`; extended per-fold details in Supplementary Section S31).

Table 12: Where graph learning helps. LOSO folds are grouped into terciles of closed-form accuracy (**Topo-QoS** ρ); system models are grouped by architectural paradigm. Mean Spearman ρ against $I^*(v)$, Application population; $\rho_{>0}$: active stratum. For system models, “learned” ranges over the pure learned engines and “closed-form” over **Topo** and **Topo-QoS**. The last column gives training-free dependency counts (Amendment 7) and dependency-graph learners (Amendment 9). Exploratory.

Regime	Where	Best choice	Evidence: raw-multigraph engines	Dependency graph
Closed-form ranks poorly	Microservices, ATM, IoT Smart City, Healthcare	Dependency count or dependency-graph learner	Topo-QoS 0.324; HGT-QoS 0.604, GAT-QoS 0.626, both 4/4 folds. The prior trims the gain (hybrids 0.502 / 0.554).	InDeg 0.693, Reach 0.700, GAT-P-QoS 0.716.
Intermediate	ESB, Telecom RAN, Financial Trading, Industrial SCADA	Dependency count or dependency-graph learner	Topo-QoS 0.561; HGT-QoS 0.589, GAT-QoS 0.660; Hybrid-HGT 0.681, Hybrid-GAT 0.697, both 4/4 folds.	InDeg 0.742, GAT-P-QoS 0.760.
Closed-form ranks well	Logistics Fleet, AV System, Enterprise, Real-Time Gaming	InDeg	Topo-QoS 0.775; HGT-QoS 0.672 (1/4 folds), GAT-QoS 0.620 (0/4); hybrids 0.786 / 0.798.	InDeg 0.858, GAT-P-QoS 0.768.
Unlike the corpus, originally pub-sub	Autoware, EdgeX, Home Assistant	Reach	Learned 0.716–0.927 vs. closed-form 0.289–0.534; learned $\rho_{>0}$ 0.183–0.833 where closed-form is negative.	Reach 0.836–0.997; $\rho_{>0}$ 0.674–0.971.
Unlike the corpus, originally RPC	Online Boutique, Train-Ticket models	Reach	Learned 0.710–0.810, but Topo 0.891 on Online Boutique; learned $\rho_{>0}$ –0.19 to +0.16.	Reach 0.966–0.998; $\rho_{>0}$ 0.813–0.976.

On the raw multigraph, learning operated primarily over node features. Three controls clarify how the raw-multigraph engines functioned. First, gradient-boosted decision trees over per-node features without graph message passing (**GBM-Feat**) achieve $\rho = 0.642$ under LOSO, level with **GAT-QoS** (0.635). Second, on the raw multigraph, all structural relations point away from Applications, meaning standard forward GAT convolutions deliver no messages to the scored nodes: deleting every edge leaves their outputs unchanged. Third, HGT reaches Applications only through reverse edge passes, and ablating them incurs no measurable loss (**HGT-QoS-U**, Section 7.2). Thus, on the raw multigraph, neural models acted as learned non-linear combiners of precomputed centralities and degree metrics rather than relational message-passing engines. On the derived dependency graph, however, directed edges flow directly from dependents to dependencies: 3-layer message passing aggregates the true dependent neighborhood, enabling **GAT-P-QoS** to outperform

GBM-Feat by +0.106 (Section 7.2).

Homogeneous versus heterogeneous architectures. Our findings provide an empirical negative result regarding relational typing in software architecture graphs. On the raw multigraph, where message passing was inactive, typing added no measurable advantage (typing main effect -0.014 , interaction $+0.001$). On the dependency graph, where message passing operates, heterogeneous transformers (HGT-P-QoS) failed to train stably (within-fold seed spread 0.208, mean $\rho = 0.514$), whereas homogeneous attention networks (GAT-P-QoS) converged reliably ($\rho = 0.748$, matching InDeg). In software architecture dependability, relational parameterization introduces optimization instability without delivering predictive gains over homogeneous aggregation.

8.3. Threats to Validity

Construct validity and oracle circularity. All ground truth in our primary experiments is simulator-derived. Because $I^*(v)$ is a reachability functional over $G_{\text{structural}}$ that propagates failure along subscriber $\rightarrow$ publisher arcs, agreement with InDeg is partly an expected property of the benchmark: any component with zero dependents trivially receives zero impact by construction. To test whether this predictive signal generalizes beyond reachability simulation, we benchmarked the rankers against the independent discrete-event queue-flow simulator (I_{dyn}) and the multi-criteria composite simulator (I_{comp}) in Section 7. The dependency counts and attention networks maintain robust rank correlations across these independently formulated oracles, demonstrating that the captured structural signals reflect genuine cascade vulnerability rather than an artifact of reachability simulation.

Internal validity and determinism. Predictors consume G_{analysis} while simulation oracles execute on $G_{\text{structural}}$, verified by continuous integration tests. Model capacity, depth, and early stopping were held constant across comparison arms. We identified that the Connectivity Degradation Index (CDI) evaluates node removal over a breadth-first sample whose tie-breaking is sensitive to Python string hashing; fixing PYTHONHASHSEED=0 ensures exact reproducibility across processes.

External validity. Synthetic topologies originate from a single generator family, and system models represent hand-authored approximations (22–41 applications) inspired by open-source projects. Evaluating architectures with tens of thousands of components will require mini-batch sampling and incremental graph caching.

Conclusion validity. All LOSO comparisons are evaluated using paired Wilcoxon signed-rank tests and bootstrap intervals. P-values are treated as nominal given shared training folds. All reported values are mechanically reconciled against released artifacts.

8.4. Limitations and Future Work

The explanation layer remains an uncalibrated design proposal for triage and has not been validated in user studies with software architects. Next steps include: (1) validating rankings against historical production outage post-mortems; (2) incorporating native RPC call semantics and backward error propagation for hybrid microservice architectures; (3) automated extraction of architecture graphs directly from Kubernetes deployment manifests and Helm charts; and (4) developing learned scoring engines on oracles that model explicit deadline penalties and buffer overflow dynamics.

9. Conclusion

Deriving explicit dependencies from architecture models is what makes simulated cascading-failure impact in publish–subscribe systems rankable before deployment. SaG turns an architecture into a typed multigraph and, through publish–subscribe rules, into an explicit graph of logical dependencies that exposes the paths along which failures propagate. On twelve held-out synthetic architectures, counting a component’s dependents on that graph (InDeg, which formalizes publish–subscribe afferent coupling) ranks simulated cascade impact at $\rho = 0.764$, beating closed-form centrality on every fold ($+0.211$). On independently authored models inspired by five open-source systems, counting transitive dependents reaches $\rho = 0.938$ with no training at all. The derived graph also lifts neural models: graph attention networks gain $+0.08$ to $+0.11$

when reading the dependency graph instead of the raw multigraph, reaching $\rho = 0.748$, level with dependency counts. In contrast, heterogeneous transformers fail to train stably on the projection, and relation typing adds no value under matched capacity.

Our independent evaluation across behavioral queue-flow simulation (I_{dyn}) and multi-criteria failure simulation (I_{comp}) demonstrates that dependency counting maintains predictive validity without relying on the construction assumptions of the reachability cascade simulator. Furthermore, feature extraction for learned models incurs significant computational overhead compared to direct in-process simulation, whereas dependency counting executes in milliseconds. SaG thus establishes an empirical negative-result benchmark for deep graph learning in architectural dependability, showing that practitioners gain most from formalizing publish–subscribe afferent coupling directly from configuration manifests.

For practice, simple dependent counting serves as an ideal pre-deployment CI gate, while SaG’s ISO/IEC 25010 explanation layer provides structured attribution to guide architectural remediation. Future work includes expanding native RPC modeling for hybrid architectures and validating ranking against production incident post-mortems.

Declarations

CRedit Authorship Contribution Statement. **Ibrahim Onuralp Yigit:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision. **Declaration of Competing Interest.** The authors declare no competing financial interests or personal relationships that could have influenced this work. **Funding.** This research received no external grant. **Data Availability.** The replication package (datasets, harnesses, checkpoints, scripts) is available on Zenodo under DOI 10.5281/zenodo.14922108 [97] with `uv/pip` environments. The public repository documents every experiment reported here — protocol, hyperparameters, reproduction command, artifacts and extended results — at <https://github.com/onuralpyigit/software-as-a-graph/tree/main/docs/research/jss/experiments>. Synthetic datasets regenerate byte-identically. The deposit ships all artifacts backing reported tables as a dated bundle (`SaG_JSS_Results_<stamp>`) with a `MANIFEST.json` recording SHA-256 digests, commit hashes, and corpus provenance. Four supplementary artifacts predate provenance stamping and carry no commit or corpus digest: `atm_scale_sweep_v3.json` (S6), `qos_label_ablation.json` (Section 4.3), `threshold_sensitivity_v3.json` (S3) and `topic_weight_sensitivity_v3.json` (S1); their correspondence to the corpus is asserted by the bundle rather than recorded in the file. The verification script (`reproduce/reconcile_manuscript.py`) runs standalone against the deposit, mechanically verifying 511 reported figures in the manuscript and supplement against the JSON artifacts.

Declaration of Generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Anthropic’s Claude to assist with typesetting, LaTeX formatting, and the development of analysis and reporting scripts in the replication package. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The study design, the choice of experiments, the interpretation of results, and all scientific claims are the authors’ own.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (accessed 9 September 2026) (2019).

- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O'Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [8] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [9] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [10] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [11] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [12] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012.
- [13] D. E. Perry, A. L. Wolf, Foundations for the study of software architecture, *ACM SIGSOFT Software Engineering Notes* 17 (4) (1992) 40–52.
- [14] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: *Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR)*, 2009, pp. 255–258.
- [15] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: *Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS)*, 1998, pp. 68–78.
- [16] SonarSource, Clean as you code, SonarQube documentation, <https://docs.sonarsource.com/sonarqube-server/latest/core-concepts/clean-as-you-code/introduction/> (accessed 9 September 2026) (2024).
- [17] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [18] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [19] C. Meiklejohn, R. Sharma, H. Miller, Service-level fault injection testing, in: *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*, ACM, 2021, pp. 388–402. doi:10.1145/3472883.3486997.
- [20] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, X. Li, Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments, in: *Proceedings of The Web Conference 2021 (WWW '21)*, ACM, 2021, pp. 3087–3098. doi:10.1145/3442381.3449905.
- [21] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.
- [22] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [23] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish–subscribe systems, in: *Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE)*, 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
- [24] C. Calero, M. Piattini (Eds.), *Green in Software Engineering*, Springer, Cham, Switzerland, 2015. doi:10.1007/978-3-319-08581-4.
- [25] L. Lannelongue, J. Grealey, M. Inouye, Green algorithms: Quantifying the carbon footprint of computation, *Advanced Science* 8 (12) (2021) 2100707. doi:10.1002/adv.202100707.
- [26] R. Verdecchia, J. Sallou, L. Cruz, A systematic review of Green AI, *WIREs Data Mining and Knowledge Discovery* 13 (4) (2023) e1507. doi:10.1002/widm.1507.
- [27] S. M. Yacoub, H. H. Ammar, A methodology for architecture-level reliability risk analysis, *IEEE Transactions on Software Engineering* 28 (6) (2002) 529–547. doi:10.1109/TSE.2002.1010058.
- [28] R. C. Cheung, A user-oriented software reliability model, *IEEE Transactions on Software Engineering SE-6* (2) (1980) 118–125.
- [29] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, *Performance Evaluation* 45 (2–3) (2001) 179–204.
- [30] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, *Software and Systems Modeling* 7 (1) (2008) 49–65.
- [31] S. Becker, H. Koziol, R. Reussner, The Palladio component model for model-driven performance prediction, *Journal of Systems and Software* 82 (1) (2009) 3–22.
- [32] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, *IEEE Transactions on Software Engineering* 35 (2) (2009) 148–161.
- [33] J. Delange, P. H. Feiler, Architecture fault modeling with the AADL error-model annex, in: *2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA)*, IEEE, 2014, pp. 361–368. doi:10.1109/SEAA.2014.20.
- [34] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [35] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [36] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020.
- [37] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: *Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS)*, 2021.
- [38] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2022.
- [39] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2023.
- [40] X. Meng, P. Shen, Y. Sun, D. Liu, J. Lu, S. Zhang, D. Pei, Microcause: Root cause analysis for microservice systems through graph neural networks, in: *Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2020, pp. 403–414.
- [41] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, D. Pei, Failure diagnosis in microservice systems: A comprehensive survey and analysis, *ACM Transactions on Software Engineering and Methodology* (2025).

- doi:10.1145/3715005.
- [42] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260.
 - [43] S. A. Bohner, R. S. Arnold, *Software Change Impact Analysis*, IEEE Computer Society Press, Los Alamitos, CA, 1996.
 - [44] S. Esparrachiar, T. Reilly, A. Rentz, Tracking and controlling microservice dependencies, *ACM Queue* 16 (4) (2018). doi:10.1145/3277539.3277541.
 - [45] X. Yang, K. Tang, X. Yao, A learning-to-rank approach to software defect prediction, *IEEE Transactions on Reliability* 64 (1) (2015) 234–246.
 - [46] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering SE-2* (4) (1976) 308–320.
 - [47] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd Edition, CRC Press, 2014.
 - [48] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, *IEEE Transactions on Software Engineering* 22 (10) (1996) 751–761.
 - [49] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: *Proc. 27th Int. Conf. on Software Engineering (ICSE)*, 2005, pp. 580–586.
 - [50] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: *Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE)*, 2007.
 - [51] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, *IEEE Transactions on Software Engineering* 33 (1) (2007) 2–13.
 - [52] T. Zimmermann, N. Nagappan, Predicting defects using network analysis on dependency graphs, in: *Proceedings of the 30th International Conference on Software Engineering (ICSE '08)*, ACM, 2008, pp. 531–540. doi:10.1145/1368088.1368161.
 - [53] R. Premraj, K. Herzig, Network versus code metrics to predict defects: A replication study, in: *2011 International Symposium on Empirical Software Engineering and Measurement (ESEM)*, IEEE, 2011, pp. 215–224. doi:10.1109/ESEM.2011.30.
 - [54] V. Bushong, D. Das, A. Al Maruf, T. Cerny, Using static analysis to address microservice architecture reconstruction, in: *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2021. doi:10.1109/ASE51524.2021.9678749.
 - [55] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, D. Taibi, Comparison of static analysis architecture recovery tools for microservice applications, *arXiv preprint* (2024). arXiv:2412.08352, doi:10.48550/arXiv.2412.08352.
 - [56] A. Santos, A. Cunha, N. Macedo, Statistical and model-driven static analysis of ROS systems, *IEEE Transactions on Software Engineering* 47 (10) (2019) 2200–2218.
 - [57] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
 - [58] D. Rud, A. Schmietendorf, R. R. Dumke, Product metrics for service-oriented infrastructures, in: *Applied Software Measurement: Proceedings of the International Workshop on Software Metrics and DASMA Software Metrik Kongress (IWSM/MetriKon 2006)*, Shaker Verlag, Aachen, Germany, 2006, pp. 161–174.
 - [59] J. Bogner, S. Wagner, A. Zimmermann, Automatically measuring the maintainability of service- and microservice-based systems: A literature review, in: *Proceedings of the 27th International Workshop on Software Measurement and 12th International Conference on Software Process and Product Measurement (IWSM Mensura '17)*, ACM, 2017, pp. 107–115. doi:10.1145/3143434.3143443.
 - [60] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: *Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA)*, LNCS 5581, 2009, pp. 146–162.
 - [61] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, *IEEE Software* 35 (3) (2018) 56–62.
 - [62] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *Journal of Systems and Software* 101 (2015) 193–220.
 - [63] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
 - [64] L. Chen, Continuous delivery: Huge benefits, but challenges too, *IEEE Software* 32 (2) (2015) 50–54.
 - [65] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).
 - [66] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).
 - [67] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
 - [68] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).
 - [69] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*, McGraw-Hill, 1980.
 - [70] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems* 30 (1–7) (1998) 107–117.
 - [71] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.
 - [72] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
 - [73] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568.
 - [74] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
 - [75] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
 - [76] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 1024–1034.
 - [77] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.

- [78] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: Proc. European Semantic Web Conference (ESWC), 2018, pp. 593–607.
- [79] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: Proc. The Web Conference (WWW), 2019, pp. 2022–2032.
- [80] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.
- [81] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: Proc. The Web Conference (WWW), 2020, pp. 2331–2341.
- [82] G. Khodabandeh, A. Ezaz, M. Babaei, N. Ezzati-Jivan, Utilizing graph neural networks for effective link prediction in microservice architectures, in: Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE), 2025.
- [83] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 9244–9255.
- [84] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 19620–19631.
- [85] W. Fu, T. Menzies, Easy over hard: A case study on deep learning for software engineering, in: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017), ACM, 2017, pp. 49–60. doi: 10.1145/3106237.3106249.
- [86] M. Ferrari Dacrema, P. Cremonesi, D. Jannach, Are we really making much progress? a worrying analysis of recent neural recommendation approaches, in: Proceedings of the 13th ACM Conference on Recommender Systems (RecSys '19), ACM, 2019, pp. 101–109. doi:10.1145/3298689.3347058.
- [87] F. Errica, M. Podda, D. Bacciu, A. Micheli, A fair comparison of graph neural networks for graph classification, in: International Conference on Learning Representations (ICLR), 2020. URL <https://openreview.net/forum?id=HygDF6NFPB>
- [88] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference, Morgan Kaufmann, 1988.
- [89] G. Beliakov, A. Pradera, T. Calvo, Aggregation functions: A guide for practitioners, Studies in Fuzziness and Soft Computing 221 (2007).
- [90] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, IEEE Transactions on Systems, Man, and Cybernetics 18 (1) (1988) 183–190.
- [91] G. H. Hardy, J. E. Littlewood, G. Pólya, Inequalities, 2nd Edition, Cambridge University Press, 1952.
- [92] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
- [93] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.
- [94] Team SimPy, Simpy: Discrete event simulation for Python, Software, <https://simpy.readthedocs.io> (accessed 9 September 2026) (2020).
- [95] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.
- [96] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.
- [97] I. O. Yigit, F. Buzluca, [dataset] software-as-a-graph: Replication package (datasets, generator configurations, simulation harnesses, model checkpoints, and analysis scripts), <https://doi.org/10.5281/zenodo.14922108> (2026). doi:10.5281/zenodo.14922108.